

DA219B – Fullstack Lab

React · Express · MongoDB · Cloud Deployment

In this lab you will design, build, and **defend** a fullstack application using a modern JavaScript stack. The focus is on genuine understanding: you must be able to explain every part of your code, show a realistic Git history, and demonstrate live changes during the seminar.

1. Project Setup

Req.	Requirement
1	Use React (Vite) for the frontend.
2	Use Express.js for the backend API.
3	Use MongoDB Atlas as your cloud database.
4	Store all secrets in a .env file. Hardcoded credentials will result in automatic failure.
5	Use concurrently (or similar) so both frontend and backend start with one command.
6	Your project must be deployable: include a README with clear setup instructions so another developer can clone and run it in under 5 minutes.

2. Application Design (Personal Requirement)

Choose a domain that is **genuinely yours**. Generic applications that match common online tutorials will not be accepted. Your application must reflect a real interest, problem, or dataset.

👉 Think: *What problem does my app solve?*

Examples of good ideas:

- Gym tracker (track workouts, sets, progress)
- Student project manager
- Recipe planner with nutrition data
- Movie collection with ratings + reviews

✗ Bad example:

- Basic todo app from YouTube

Req.	Requirement
1	Seed the database with realistic, original data — not placeholder strings like 'item1', 'test@test.com', or Lorem Ipsum.
2	Add at least one custom field that reflects your specific domain (not just name / description / price).
3	Be ready to explain your design decisions — why this domain, why this schema, why these relationships — during the seminar.
4	State the problem your app solves in one sentence in your README.

✓ Example (Gym app):

```
{
  "exercise": "Bench Press",
  "sets": 4,
  "reps": 10,
  "muscleGroup": "Chest" // custom field
}
```

🎯 What The teacher verify

- Your seed data must look realistic. Teacher will check whether names, values, and relationships make sense for the domain.
- You will be asked to add a new field or document live during the seminar — your data model must support this.

3. Database

Req.	Requirement
1	Create at least 3 collections with meaningful names for your domain.
2	Seed at least 5 realistic documents per collection.
3	Define and implement relationships between collections using ObjectId references.
4	Include a data model diagram (ERD or equivalent) in your report that matches your actual schema.
5	Add Mongoose validation (required fields, types, min/max) to at least one schema.

Example:

Collections:

- users
- workouts
- exercises

Relationship:

workout.userId → users._id

4. Backend API

Req.	Requirement
1	Implement full CRUD for your main entity (POST, GET, PUT/PATCH, DELETE). Example routes: POST /api/workouts GET /api/workouts PUT /api/workouts/:id DELETE /api/workouts/:id
2	Implement at least 2 relational endpoints that join or reference multiple collections.
3	Create one custom endpoint — filtering, aggregation, or statistics — that makes sense for your domain. /api/workouts?user=123 → filter /api/stats/total-reps → statistics
4	Handle errors consistently: return appropriate HTTP status codes (400, 404, 409, 500) with a JSON error message body.

5	Organise your backend using the Router → Controller → Model pattern. All logic in a single file is not acceptable.
6	Write input validation on POST and PUT routes. Do not trust the client.

🔗 What Teacher verify

- You must explain how one route works end-to-end (request → middleware → controller → DB → response) during the seminar.
- Teacher may ask you to add a new query parameter to a route live. You must be able to do this.

5. Frontend (React)

Req.	Requirement
1	Display your main entity data in a table or structured list .
2	Include a form to create new records with controlled inputs.
3	Implement update (inline or modal edit) and delete with confirmation.
4	Add column sorting via clickable headers. (OPTIONAL)
5	Implement auto-refresh with setInterval — and clean it up properly on unmount using the useEffect return function.
6	Add one custom interactive feature : search, filter, pagination, or similar.
7	Use at least 3 separate components . A single monolithic App.jsx is not acceptable. Each component must have a clear, single responsibility. Example components: • WorkoutList.jsx • WorkoutForm.jsx • WorkoutRow.jsx
8	Display loading and error states — the UI must not be blank while fetching or silent when a request fails.

🔗 What Teacher verify

- You will be asked to explain one component's props and state during the seminar.
- Teacher may ask you to add a new filter or column live. Your component structure must allow this.
- useEffect cleanup (clearing setInterval) is a common interview question — know why it matters.

6. Git Requirements

Git history is treated as a **development diary**. It must tell a believable story of how your project grew — not appear as a single dump of finished code.

Req.	Requirement
1	Minimum 10 meaningful commits spread across the project timeline.
2	Commit messages must follow the format type: short description — e.g. feat: add DELETE route for recipes, fix: clear interval on unmount.

3	Do not submit a single final commit or squash your history.
4	At least one commit must show an iteration : a feature that was first implemented one way and then improved. Reference it in your report.

✓ Good:

feat: add POST route for workouts

fix: handle missing userId

✗ Bad:

done

final

update

🎯 What Teacher verify

- A flat commit history (all commits on the same day, or fewer than 5 commits) is grounds for failure.
- Commit messages like 'final', 'done', or 'update' will be flagged.
- Teacher will look at the diff of individual commits — they must show incremental, logical changes.

7. Report Requirements (PDF submission)

Req.	Requirement
1	System overview (5–8 lines): what the app does and who it is for.
2	Database design : include your ERD and explain each collection and relationship.
3	Two API endpoints : show the route, method, expected request body, and example response.
4	Reflection : one challenge you faced, how you solved it, and what you learned.
5	Feature iteration : describe one feature that changed during development. Reference at least two commit hashes from your Git history.

8. Seminar / Presentation

The seminar tests whether you understand your own code. You must be able to make **live changes on demand**. Prepare your environment beforehand — a project that cannot be run during the seminar cannot be assessed.

Note that you might not be asked to do all but be prepared for all the following!

Req.	Requirement
1	Demo the running application: show all CRUD operations in the UI.
2	Explain one full route end-to-end: from HTTP request to DB query to response.
3	Explain one component : its props, its state, and why you structured it this way.
4	Explain one Mongoose schema : fields, types, validation, and relationships.
5	Make a live change requested by the assessor — e.g. add a query parameter, add a field, or change a validation rule. The change must work.
6	Walk through two commits in your Git history and explain what changed and why.

7	Answer follow-up questions about your design decisions.
---	--

 What Teacher verify

- If you cannot explain a piece of code when asked, it will be treated as if it does not exist for grading purposes.
- The live change is intentionally unpredictable — you must understand your codebase, not just memorise it.
- Referring to 'I found this online' without being able to explain it is not acceptable.

9. Grading Summary

Your grade is based on four equally weighted dimensions:

Dimension	What earns full marks	Common failure reasons
Functionality (25%)	All CRUD works. API handles errors. UI shows loading/error states. Custom feature works.	Broken routes. Silent failures. Placeholder data.
Code Quality (25%)	Router/Controller/Model separation. Validation present. Components single-responsibility. No hardcoded secrets.	Monolithic files. No validation. Credentials in code.
Git History (25%)	10+ commits spread over time. Conventional messages. Visible iteration.	Single commit. Vague messages. All commits same day.
Seminar (25%)	Can explain all parts. Makes live change successfully. Answers follow-up questions confidently.	Cannot explain code. Live change fails. Reads from notes only.

10. Important Rules

Automatic failure conditions

- Hardcoded credentials or secrets committed to Git.
- A flat Git history (all commits on one day, or fewer than 5 commits).
- Unable to run the project during the seminar.
- Unable to explain any part of the submitted code when asked.
- Seed data that is clearly placeholder (Lorem Ipsum, test@test.com, item1/item2/item3).

Strong advice

- Build incrementally — commit after every working feature, not at the end.
- Test your API with Bruno or Postman before connecting the frontend.
- Write your README as you go, not the night before submission.
- If you used a tutorial or Stack Overflow answer, understand it before submitting — you will be asked about it.
- Practice the live-change scenario: ask a classmate to pick a random small change and time yourself.